

计算机系统设计PA3实验报告

苏耀磊 2311727

一、实验目的和环境概述

- 掌握在 Nanos-lite 中实现用户程序加载、系统调用与设备抽象的基本方法，建立用户态程序、内核服务与底层硬件之间的对应关系。
- 理解文件系统在操作系统中的作用，学会将 ramdisk 中的普通文件以及 /dev/fb、/dev/events、/proc/dispinfo 等特殊设备抽象成统一的文件接口。
- 掌握 brk/sbrk、write/read/lseek/open/close 等常见系统调用的实现思路，理解 libc、libos 与内核 syscall 分发之间的协作方式。
- 通过实现键盘、时钟等输入事件的文本化封装，理解事件驱动模型及设备输入的读取优先级，提升对操作系统 I/O 处理流程的整体认识。
- 最终能够成功在系统上运行仙剑奇侠传。
- 综合运用体系结构、操作系统与调试知识，分析并解决实验中出现的编译错误、格式化输出异常和设备交互等问题。

本次实验环境使用 vscode 连接 docker 容器，在 debian 中进行实验，实验指导书参考 ics2017。

二、实验内容

- 计算机系统设计PA3实验报告
 - 一、实验目的和环境概述
 - 二、实验内容
 - 三、实验过程
 - 1.实现loader
 - 2.实现中断机制
 - 3.重新组织TrapFrame结构体
 - 问题：对比异常与函数调用
 - 问题：诡异的代码
 - 4.实现系统调用
 - 5.在Nanos-lite上运行Hello world
 - 6.实现堆区管理
 - 7.让loader使用文件
 - 8.实现完整的文件系统
 - 9.把VGA显存抽象成文件
 - 10.把设备输入抽象成文件
 - 11.在NEMU中运行仙剑奇侠传
 - 问题：不再神秘的秘技
 - 四、必答题
 - 游戏存档读取过程
 - 屏幕更新过程
 - 五、实验过程中的问题与解决方法
-

三、实验过程

1.实现loader

首先在 navy-apps/Makefile.check 里，我们修改编译选项，让 Navy-apps 项目上的程序默认编译到 x86 中：

```
1  ISA ?= x86
```

然后在 navy-apps/tests/dummy 下执行

```
make
```

在 nanos-lite/ 目录下执行

```
make update
```

生成 ramdisk 镜像文件。

根据指导书，loader 是一个用于加载程序的模块，我们知道程序中包括代码和数据，它们都是存储在可执行文件中。加载的过程就是把可执行文件中的代码和数据放置在正确的内存位置，然后跳转到程序入口，程序就开始执行了。在这里，就是将 ramdisk 中从 0 开始的所有内容放置在 0x4000000，并把这个地址作为程序的入口返回即可。

在 nanos-lite/src/loader.c 里，

```
5  extern void ramdisk_read(void *buf, off_t offset, size_t len);
6  extern size_t get_ramdisk_size();
7
8  uintptr_t loader(_Protect *as, const char *filename) {
9      ramdisk_read(DEFAULT_ENTRY, 0, get_ramdisk_size());
10     return (uintptr_t)DEFAULT_ENTRY;
11 }
```

使用 ramdisk_read(void *buf, off_t offset, size_t len) 将 ramdisk 中 offset 偏移处的 len 字节读入到 buf 中，get_ramdisk_size() 用来返回 ramdisk 的大小，单位为字节。

之后我们在 nanos-lite/ 目录下执行

```
make run
```

即可运行带有最新版 ramdisk 的 Nanos-lite。

2.实现中断机制

首先，我们需要在 nemu/include/cpu/reg.h 里添加 IDTR 寄存器，

```

35     struct {
36         uint16_t limit;
37         uint32_t base;
38     } idtr;

```

然后需要添加 `lidt` 和 `int` 指令，这个和 PA2 的内容很相似了，首先我们在 `nemu/src/cpu/exec/system.c` 里添加两条指令的 `make_EHelper()` 函数：

```

7     make_EHelper(lidt) {
8         rtlreg_t base = id_dest->addr;
9         rtlreg_t limit = vaddr_read(base, 2);
10        rtlreg_t ptr = vaddr_read(base + 2, 4);
11
12        cpu.idtr.limit = limit;
13        cpu.idtr.base = ptr;
14
15        print_asm_template1(lidt);
16    }

```

```

34    make_EHelper(int) {
35        raise_intr(id_dest->val & 0xff, *eip);
36        decoding.is_jump = 1;
37        decoding.jump_eip = cpu.eip;
38
39        print_asm("int %s", id_dest->str);
40
41    #ifdef DIFF_TEST
42        diff_test_skip_nemu();
43    #endif
44    }

```

这里对于 `int`，我们需要实现 `raise_intr()` 函数：

```

3
4 void raise_intr(uint8_t NO, vaddr_t ret_addr) {
5     /* TODO: Trigger an interrupt/exception with ``NO``.
6      * That is, use ``NO`` to index the IDT.
7      */
8
9     rtlreg_t idt_base = cpu.idtr.base;
10    rtlreg_t desc_lo = vaddr_read(idt_base + NO * 8, 4);
11    rtlreg_t desc_hi = vaddr_read(idt_base + NO * 8 + 4, 4);
12    vaddr_t target = (desc_hi & 0xffff0000) | (desc_lo & 0xffff);
13
14    rtl_push(&cpu.eflags);
15    rtl_li(&t0, 0x8);
16    rtl_push(&t0);
17    rtl_li(&t0, ret_addr);
18    rtl_push(&t0);
19
20    cpu.eip = target;
21 }

```

raise_intr() 函数是用来模拟 x86 的中断触发过程，首先根据中断号 NO 在 IDT 中找到对应的中断描述符，并从描述符中解析出中断处理程序的入口地址 target。随后，函数依次将当前 CPU 的 EFLAGS、代码段寄存器 CS（这里固定为 0x8）以及返回地址 ret_addr 压入栈中，用于中断结束后的现场恢复。最后，通过 cpu.eip = target 将 CPU 的执行流跳转到对应的中断处理程序，从而完成一次中断进入过程。

之后在 nemu/src/cpu/exec/exec.c 补充如下内容：

```

68 make_group(gp7,
69     EMPTY, EMPTY, EMPTY, EX(lidt),
70     EMPTY, EMPTY, EMPTY, EMPTY)

126 /* 0xc8 */ EMPTY, EX(leave), EMPTY, EMPTY,
127 /* 0xcc */ EMPTY, IDEXW(I, int, 1), EMPTY, EX(iret),

```

最后将声明添加到 nemu/src/cpu/exec/all-instr.h 里：

```

47 make_EHelper(lidt);
48 make_EHelper(int);

```

3.重新组织TrapFrame结构体

首先，我们首先需要实现 pusha 指令，在 nemu/src/cpu/exec/data-mov.c 里我们补全它的定义：

```

21  make_EHelper(pusha) {
22      rtlreg_t temp_esp = cpu.esp;
23
24      rtl_push(&cpu.eax);
25      rtl_push(&cpu.ecx);
26      rtl_push(&cpu.edx);
27      rtl_push(&cpu.ebx);
28      rtl_push(&temp_esp);
29      rtl_push(&cpu.ebp);
30      rtl_push(&cpu.esi);
31      rtl_push(&cpu.edi);
32
33      print_asm("pusha");
34  }

```

顺便把 popa 指令也一并实现：

```

36  ✓ make_EHelper(popa) {
37      rtl_pop(&t0);
38      rtl_sr_l(R_EDI, &t0);
39      rtl_pop(&t0);
40      rtl_sr_l(R_ESI, &t0);
41      rtl_pop(&t0);
42      rtl_sr_l(R_EBP, &t0);
43      rtl_pop(&t0);
44      rtl_pop(&t0);
45      rtl_sr_l(R_EBX, &t0);
46      rtl_pop(&t0);
47      rtl_sr_l(R_EDX, &t0);
48      rtl_pop(&t0);
49      rtl_sr_l(R_ECX, &t0);
50      rtl_pop(&t0);
51      rtl_sr_l(R_EAX, &t0);
52
53      print_asm("popa");
54  }

```

根据指导书的内容，接下来需要重新组织 nexus-am/am/arch/x86-nemu/include/arch.h 中定义的 _RegSet 结构体的成员，使得这些成员声明的顺序和 nexus-am/am/arch/x86-nemu/src/trap.S 中构造的 trap frame 保持一致：

```

9  struct _RegSet {
10      uintptr_t edi, esi, ebp, esp, ebx, edx, ecx, eax;
11      int      irq;
12      uintptr_t error_code, eip, cs, eflags;
13  };

```

问题：对比异常与函数调用

我们知道进行函数调用的时候也需要保存调用者的状态：返回地址，以及调用约定(calling convention)中需要调用者保存的寄存器。而进行异常处理之前却要保存更多的信息。尝试对比它们，并思考两者保存信息不同是什么原因造成的。

在普通的函数调用中，保存调用者状态的主要目的是为了在函数执行结束后能够正确返回。通常需要保存的信息包括返回地址以及调用约定中规定的由调用者保存的寄存器，这些寄存器中存放了函数调用前的临时数据和参数传递信息，保存这些信息的目的是保证被调用函数执行时不会破坏调用者的环境，从而在函数返回时能够恢复原状态继续执行。

而在异常或中断处理之前，需要保存的信息更加全面，除了返回地址和部分寄存器外，还必须保存 CPU 的状态标志寄存器、当前的代码段寄存器，以及在特权级切换时可能还要保存栈指针、段寄存器等。这是因为异常或中断可能打断任何正在执行的代码，包括操作系统内核或用户程序，并且处理程序可能运行在不同的特权级上。如果不保存完整的 CPU 状态，中断处理完成后无法准确恢复被打断程序的执行环境，可能导致程序行为异常或者系统崩溃。

问题：诡异的代码

trap.S中有一行pushl %esp的代码,乍看之下其行为十分诡异.你能结合前后的代码理解它的行为吗? Hint: 不用想太多, 其实都是你学过的知识.

```
#----|-----entry-----|-----errorcode---|---irq id---|---handler---|
.globl vecsys;    vecsys:    pushl $0;    pushl $0x80; jmp asm_trap
.globl vecnull;   vecnull:   pushl $0;    pushl $-1; jmp asm_trap

asm_trap:
    pushal

    pushl %esp
    call irq_handle

    addl $4, %esp

    popal
    addl $8, %esp

    iret
```

这一行代码的作用就是将当前中断现场在栈中的地址作为参数传递给 irq_handle。

在进入 asm_trap 之前，程序已经先通过 pushl 0和pushl 0x80 等指令将错误码和中断号压入栈中；随后 pushal 又会自动把所有通用寄存器依次压栈。因此，当执行到 pushl %esp 时，当前的 esp 已经指向了一整块完整保存好的中断上下文区域。

接下来执行 call irq_handle 时，根据 x86 的函数调用约定，之前压入栈中的 esp 值会作为 irq_handle 的第一个参数传入，也就是说，irq_handle 接收到的实际上是一个指向当前中断现场的指针，这样，C 语言中的中断处理函数就能够通过这个指针直接访问和修改被保存的寄存器、中断号、错误码以及返回地址等信息。

4.实现系统调用

首先，我们需要在 do_event() 中识别出系统调用事件 _EVENT_SYSCALL，然后调用 do_syscall()，在 nanos-lite/src/irq.c 里：

```

6  static _RegSet* do_event(_Event e, _RegSet* r) {
7      switch (e.event) {
8          case _EVENT_SYSCALL: return do_syscall(r);
9          default: panic("Unhandled event ID = %d", e.event);
10     }

```

之后在 nexus-am/am/arch/x86-nemu/include/arch.h 中实现正确的 SYSCALL_ARGx() 宏，让它们从作为参数的现场 reg 中获得正确的系统调用参数寄存器：

```

15  #define SYSCALL_ARG1(r) ((r)->eax)
16  #define SYSCALL_ARG2(r) ((r)->ebx)
17  #define SYSCALL_ARG3(r) ((r)->ecx)
18  #define SYSCALL_ARG4(r) ((r)->edx)

```

之后需要添加 SYS_none 系统调用，在 nanos-lite/src/syscall.c 里：

```

5  _RegSet* do_syscall(_RegSet *r) {
6      uintptr_t a[4];
7      a[0] = SYSCALL_ARG1(r);
8      a[1] = SYSCALL_ARG2(r);
9      a[2] = SYSCALL_ARG3(r);
10     a[3] = SYSCALL_ARG4(r);
11
12     switch (a[0]) {
13         case SYS_none:
14             SYSCALL_ARG1(r) = 1;
15             return r;

```

这里 do_syscall() 函数用于处理用户程序发起的系统调用，首先通过 SYSCALL_ARG 这些从寄存器中取出系统调用的参数，并保存到数组 a 中，随后通过 switch(a[0]) 根据系统调用号分发到不同的处理逻辑。当系统调用号为 SYS_none 时，函数将返回值寄存器设置为 1，也就是 SYSCALL_ARG1(r) = 1，最后返回当前寄存器现场 r。

接下来需要实现 iret 指令，在 nemu/src/cpu/exec/system.c 里补充 make_EHelper() 函数：

```

46  make_EHelper(iret) {
47      rtl_pop(&t0);
48      rtl_pop(&t1);
49      rtl_pop(&t2);
50      decoding.is_jump = 1;
51      decoding.jump_eip = t0;
52      cpu.eflags = t2;
53
54      print_asm("iret");
55  }

```

以及 nemu/src/cpu/exec/exec.c 里：

```

127  /* 0xcc */ EMPTY, IDEXW(I, int, 1), EMPTY, EX(iret),

```

并补充相应的声明到 nemu/src/cpu/exec/all-instr.h 里：

```

49  make_EHelper(iret);

```

最后实现 SYS_exit 系统调用，同样是刚才的 nanos-lite/src/syscall.c 里，调用 _halt()：

```
34     case SYS_exit:
35         _halt(a[1]);
36         return NULL;
```

完成上面的内容，我们就可以运行 dummy 程序了，在 nanos-lite/ 目录下执行

```
make run
```

执行结果如下：

```
SuYaolei@988d7f56373a:~/code/ics2017/nanos-lite$ make run
+ CC src/cpu/reg.c
+ LD build/nemu
./build/nemu -l /home/SuYaolei/code/ics2017/nanos-lite/build/nemu-log.txt /home/SuYaolei/code/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
[src/monitor/monitor.c,65,load_img] The image is /home/SuYaolei/code/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 17:10:28, May 17 2026
For help, type "help"
(nemu) c
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 17:10:27, May 17 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x102480, end = 0x1d510c5, size = 29682757 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
nemu: HIT GOOD TRAP at eip = 0x00100032

(nemu) █
```

看到 GOOD TRAP 的信息，我们的实现没有问题。

5.在Nanos-lite上运行Hello world

我们需要实现 write() 系统调用，同样是上面的文件里：

```
15     case SYS_write:
16         // Log("SYS_write fd=%d count=%d buf=%p", (int)a[1], (int)a[3], (void *)a[2]);
17         if (a[1] == 1 || a[1] == 2) {
18             const char *buf = (const char *)a[2];
19             for (size_t i = 0; i < a[3]; i++) {
20                 _putc(buf[i]);
21             }
22             SYSCALL_ARG1(r) = a[3];
23             return r;
24         }
25         SYSCALL_ARG1(r) = (uintptr_t)-1;
26         return r;
```

首先判断文件描述符 a[1] 是否为 1 或 2，分别对应标准输出 stdout 和标准错误输出 stderr。如果满足条件，就将 a[2] 作为字符缓冲区地址，将 a[3] 作为输出长度，然后通过循环逐字符调用 _putc(buf[i]) 输出内容。输出完成后，将实际写入的字节数 a[3] 作为返回值写回寄存器 SYSCALL_ARG1(r)，表示写入成功。若文件描述符不是 1 或 2，则说明当前系统暂不支持写入其他文件，此时返回 -1 表示系统调用失败。

之后在 navy-apps/libs/libos/src/nanos.c 的 _write() 中调用系统调用接口函数：

```

30  int _write(int fd, void *buf, size_t count){
31      return _syscall_(SYS_write, fd, (uintptr_t)buf, count);
32  }

```

6.实现堆区管理

需要在 Nanos-lite 中实现 SYS_brk 系统调用，这个实现比较简单，如下：

```

31  case SYS_brk:
32      SYSCALL_ARG1(r) = 0;
33      return r;

```

然后在用户层实现 _sbrk(), 在 navy-apps/libs/libos/src/nanos.c 里：

```

34  void *_sbrk(intptr_t increment){
35      static uintptr_t program_brk = (uintptr_t)&_end;
36      uintptr_t old_brk = program_brk;
37      uintptr_t new_brk = old_brk + increment;
38
39      if (_syscall_(SYS_brk, new_brk, 0, 0) == 0) {
40          program_brk = new_brk;
41          return (void *)old_brk;
42      }
43
44      return (void *)-1;
45  }

```

先用 program_brk 表示当前堆顶位置，初始值为 _end，也就是程序数据段结束的位置，先记录旧的堆顶 old_brk，再计算扩展后的新堆顶 new_brk = old_brk + increment，随后发起 brk 系统调用，请求操作系统将堆空间扩展到 new_brk。如果系统调用返回 0，说明扩展成功，此时更新 program_brk，并返回原来的堆顶地址 old_brk，作为新分配内存的起始地址；如果失败，则返回 (void *)-1 表示内存申请失败。

完成上面的内容，我们执行 hello 程序，运行结果如下：

```

SuYaoLei@988d7f56373a:~/code/ics2017/nanos-lite$ make run
(nemu) c
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 17:55:27, May 17 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x102480, end = 0x1d510c5, size = 29682757 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
Hello World!
Hello World for the 2th time
Hello World for the 3th time
Hello World for the 4th time
Hello World for the 5th time
Hello World for the 6th time
Hello World for the 7th time
Hello World for the 8th time
Hello World for the 9th time
Hello World for the 10th time
Hello World for the 11th time

```

能看到程序执行没有问题。

7.让loader使用文件

先对 nanos-lite/Makefile 作如下修改：

之后为每一个已经打开的文件引入偏移量属性 `open_offset`，来记录目前文件操作的位置，每次对文件读写了多少个字节，偏移量就前进多少，在 `nanos-lite/src/fs.c` 里：

```
3  typedef struct {
4      char *name;
5      size_t size;
6      off_t disk_offset;
7      off_t open_offset; // 文件被打开之后的读写指针
8  } Finfo;
```

接下来需要实现 `fs_open()`，`fs_read()` 和 `fs_close()`：

```
37  int fs_open(const char *pathname, int flags, int mode) {
38      (void)flags;
39      (void)mode;
40
41      for (int fd = 0; fd < (int)NR_FILES; fd++) {
42          if (strcmp(pathname, file_table[fd].name) == 0) {
43              file_table[fd].open_offset = 0;
44              return fd;
45          }
46      }
47
48      return -1;
49  }
```

`fs_open()` 函数用来打开文件，并返回对应的文件描述符，通过遍历文件系统中的文件表 `file_table`，通过 `strcmp` 判断传入的文件名是否与文件表中的文件名一致，如果找到对应文件，就将该文件的读写偏移量 `open_offset` 重置为 0，表示从文件开头开始访问，并返回当前文件在文件表中的下标 `fd` 作为文件描述符，如果遍历结束仍未找到对应文件，则返回 -1 表示打开失败。

```

58  size_t fs_read(int fd, void *buf, size_t len) {
59  if (fd < 0 || fd >= (int)NR_FILES) {
60      return 0;
61  }
62
63  if (fd == FD_EVENTS) {
64      return events_read(buf, len);
65  }
66
67  Finfo *f = &file_table[fd];
68  size_t remain = f->size > (size_t)f->open_offset ? f->size - (size_t)f->open_offset : 0;
69  size_t nread = len < remain ? len : remain;
70
71  if (fd == FD_DISPINFO) {
72      dispinfo_read(buf, f->open_offset, nread);
73      f->open_offset += nread;
74      return nread;
75  }
76
77  ramdisk_read(buf, f->disk_offset + f->open_offset, nread);
78  f->open_offset += nread;
79  return nread;
80 }

```

fs_read() 函数用于从文件中读取数据到缓冲区 buf 中，首先检查文件描述符 fd 是否合法，随后，对于特殊文件 FD_EVENTS，函数调用 events_read 读取输入事件数据；对于 FD_DISPINFO，则调用 dispinfo_read 读取显示设备信息，并更新文件当前读写位置 open_offset，这部分内容在后面的第三阶段，需要补充 /proc/dispinfo 和 /dev/events 这些特殊文件的支持，后面不再赘述。

对于普通文件，函数先计算当前还能读取的剩余字节数 remain，再取 len 和 remain 中较小的值作为实际读取长度 nread，防止越界读取。之后通过 ramdisk_read 从 ramdisk 中读取数据到用户缓冲区，并将文件偏移量 open_offset 增加 nread，表示文件读取位置向后移动。最后返回实际读取的字节数 nread。

```

142 int fs_close(int fd) {
143     if (fd < 0 || fd >= (int)NR_FILES) {
144         return -1;
145     }
146
147     file_table[fd].open_offset = 0;
148     return 0;
149 }

```

fs_close() 函数用于关闭文件。函数首先检查文件描述符 fd 是否合法，如果超出文件表范围，则返回 -1 表示关闭失败；否则，将对应文件在文件表中的读写偏移量 open_offset 重置为 0，表示文件访问状态被清空，然后返回 0 表示关闭成功。

当然还有一个辅助函数 fs_filesz()，用于获取文件大小：

```

51  size_t fs_filesz(int fd) {
52      if (fd < 0 || fd >= (int)NR_FILES) {
53          return 0;
54      }
55      return file_table[fd].size;
56  }

```

并将这些函数的声明添加到 nanos-lite/include/fs.h 里：

```
8 void init_fs(void);
9 int fs_open(const char *pathname, int flags, int mode);
10 size_t fs_read(int fd, void *buf, size_t len);
11 size_t fs_write(int fd, const void *buf, size_t len);
12 off_t fs_lseek(int fd, off_t offset, int whence);
13 size_t fs_filesz(int fd);
14 int fs_close(int fd);
```

这里还添加了一些，后文不再赘述。

那么接下来我们就可以完成 loader()，在 loader 中使用文件名来指定加载的程序了，如下：

```
6  voidptr_t loader(_Protect *as, const char *filename) {
7      (void)as;
8      if (filename == NULL) {
9          filename = "/bin/hello";
10     }
11
12     int fd = fs_open(filename, 0, 0);
13     assert(fd >= 0);
14
15     fs_read(fd, DEFAULT_ENTRY, fs_filesz(fd));
16     fs_close(fd);
17     return (voidptr_t)DEFAULT_ENTRY;
18 }
```

8.实现完整的文件系统

接下来需要实现 fs_write()，在 nanos-lite/src/fs.c 里：

```

82  ∨ size_t fs_write(int fd, const void *buf, size_t len) {
83  ∨    if (fd < 0 || fd >= (int)NR_FILES) {
84      return 0;
85  }
86
87      Finfo *f = &file_table[fd];
88
89  ∨    if (fd == FD_STDIN || fd == FD_EVENTS) {
90      return 0;
91  }
92
93  ∨    if (fd == FD_STDOUT || fd == FD_STDERR) {
94      const char *ch = buf;
95  ∨      for (size_t i = 0; i < len; i++) {
96          _putc(ch[i]);
97      }
98      f->open_offset += len;
99      return len;
100 }
101
102 ∨ if (fd == FD_FB) {
103     fb_write(buf, f->open_offset, len);
104     f->open_offset += len;
105     return len;
106 }
107
108     size_t remain = f->size > (size_t)f->open_offset ? f->size - (size_t)f->open_offset : 0;
109     size_t nwrite = len < remain ? len : remain;
110     ramdisk_write(buf, f->disk_offset + f->open_offset, nwrite);
111     f->open_offset += nwrite;
112     return nwrite;
113 }

```

fs_write() 函数用于向文件写入数据。

首先检查文件描述符 fd 是否合法，随后根据不同类型的文件执行不同的写入操作，对于 FD_STDIN 和 FD_EVENTS，由于这些文件不支持写入，因此直接返回 0。

对于标准输出 FD_STDOUT 和标准错误输出 FD_STDERR，函数会逐字符调用 _putc 将数据输出到终端，并更新文件偏移量 open_offset。

对于帧缓冲设备 FD_FB，则调用 fb_write 将数据写入显存，实现图形输出。对于普通文件，函数先计算当前剩余可写空间 remain，再取 len 与 remain 中较小值作为实际写入长度 nwrite，防止越界写入，然后通过 ramdisk_write 将数据写入 ramdisk 中对应的位置，并更新文件偏移量。

最后返回实际写入的字节数。

接下来是 fs_lseek() 函数：

```

115  ∨ off_t fs_lseek(int fd, off_t offset, int whence) {
116  ∨    if (fd < 0 || fd >= (int)NR_FILES) {
117      | return -1;
118      }
119
120  ∨    if (fd == FD_STDIN || fd == FD_STDOUT || fd == FD_STDERR || fd == FD_EVENTS) {
121      | return -1;
122      }
123
124      Finfo *f = &file_table[fd];
125      off_t new_offset;
126
127  ∨    switch (whence) {
128      | case SEEK_SET: new_offset = offset; break;
129      | case SEEK_CUR: new_offset = f->open_offset + offset; break;
130      | case SEEK_END: new_offset = f->size + offset; break;
131      | default: return -1;
132      }
133
134  ∨    if (new_offset < 0) {
135      | return -1;
136      }
137
138      f->open_offset = new_offset;
139      return new_offset;
140  }

```

fs_lseek 函数用于修改文件的读写位置，首先检查文件描述符 fd 是否合法，同时禁止对标准输入、标准输出、标准错误以及事件设备文件进行偏移操作，如果不满足条件则返回 -1。

随后根据参数 whence 决定新的偏移位置：SEEK_SET 表示从文件开头计算偏移，SEEK_CUR 表示在当前偏移基础上移动，SEEK_END 表示从文件末尾开始计算。如果 whence 类型非法，同样返回 -1。

接着函数检查新的偏移量 new_offset 是否小于 0，防止出现非法位置；若合法，则更新文件的 open_offset，并返回新的文件位置。

更改完这些以后，我们修改 navy-apps/libs/libos/src/nanos.c 里的那些系统调用：

```

22  ∨ void _exit(int status) {
23      | _syscall_(SYS_exit, status, 0, 0);
24      }
25
26  ∨ int _open(const char *path, int flags, mode_t mode) {
27      | return _syscall_(SYS_open, (uintptr_t)path, flags, mode);
28      }
29
30  ∨ int _write(int fd, void *buf, size_t count){
31      | return _syscall_(SYS_write, fd, (uintptr_t)buf, count);
32      }

```

```

47 ∨ int _read(int fd, void *buf, size_t count) {
48     return _syscall_(SYS_read, fd, (uintptr_t)buf, count);
49 }
50
51 ∨ int _close(int fd) {
52     return _syscall_(SYS_close, fd, 0, 0);
53 }
54
55 ∨ off_t _lseek(int fd, off_t offset, int whence) {
56     return _syscall_(SYS_lseek, fd, offset, whence);
57 }

```

并在 nanos-lite/src/syscall.c 的 do_syscall() 补充相应内容：

```

12 ∨ switch (a[0]) {
13 ∨     case SYS_none:
14         SYSCALL_ARG1(r) = 1;
15         return r;
16 ∨     case SYS_open:
17         SYSCALL_ARG1(r) = fs_open((const char *)a[1], a[2], a[3]);
18         return r;
19 ∨     case SYS_read:
20         SYSCALL_ARG1(r) = fs_read(a[1], (void *)a[2], a[3]);
21         return r;
22 ∨     case SYS_close:
23         SYSCALL_ARG1(r) = fs_close(a[1]);
24         return r;
25 ∨     case SYS_write:
26         SYSCALL_ARG1(r) = fs_write(a[1], (const void *)a[2], a[3]);
27         return r;
28 ∨     case SYS_lseek:
29         SYSCALL_ARG1(r) = fs_lseek(a[1], a[2], a[3]);
30         return r;
31 ∨     case SYS_brk:
32         SYSCALL_ARG1(r) = 0;
33         return r;
34 ∨     case SYS_exit:
35         _halt(a[1]);
36         return NULL;
37     default: panic("Unhandled syscall ID = %d", a[0]);
38 }

```

运行测试程序 /bin/text，我们只需要将 nanos-lite/src/main.c 里的代码修改为用 loader 加载：

```

33 ||| uint32_t entry = loader(NULL, "/bin/text");

```

运行 make run，结果如下：

```

SuYaolei@988d7f56373a:~/code/ics2017/nanos-lite$ make run
./build/nemu -l /home/SuYaolei/code/ics2017/nanos-lite/build/nemu-log.txt /home/SuYaolei/code/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
[src/monitor/monitor.c,65,load_img] The image is /home/SuYaolei/code/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 14:41:28, May 18 2026
For help, type "help"
(nemu) c
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 14:41:26, May 18 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x102480, end = 0x1d510c5, size = 29682757 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
PASS!!!
nemu: HIT GOOD TRAP at eip = 0x00100032

(nemu) █

```

程序输出 PASS!!! 的信息，实现没有问题。

9.把VGA显存抽象成文件

首先，需要在 `init_fs()` (在 `nanos-lite/src/fs.c` 中定义) 中对文件记录表中 `/dev/fb` 的大小进行初始化，如下：

```

32 void init_fs() {
33     file_table[FD_FB].size = (size_t)_screen.width * _screen.height * sizeof(uint32_t);
34     file_table[FD_DISPINFO].size = dispinfo_size;
35 }

```

首先根据屏幕的宽度和高度计算帧缓冲设备 `FD_FB` 的大小，即 `width × height × sizeof(uint32_t)`，表示整个屏幕所需的显存字节数，并将结果保存到对应文件表项的 `size` 字段中，随后，再将显示信息文件 `FD_DISPINFO` 的大小设置为 `dispinfo_size`。

然后是实现 `fb_write()` (在 `nanos-lite/src/device.c` 中定义)，需要先从 `offset` 计算出屏幕上的坐标，然后调用 IOE 的 `_draw_rect()` 接口，把 `buf` 中的 `len` 字节写到屏幕上 `offset` 处：

```

50 void fb_write(const void *buf, off_t offset, size_t len) {
51     size_t pixels = len / sizeof(uint32_t);
52     const uint32_t *p = buf;
53     int x = (offset / sizeof(uint32_t)) % _screen.width;
54     int y = (offset / sizeof(uint32_t)) / _screen.width;
55
56     while (pixels > 0) {
57         int w = _screen.width - x;
58         if (w > (int)pixels) {
59             w = (int)pixels;
60         }
61
62         _draw_rect(p, x, y, w, 1);
63         p += w;
64         pixels -= w;
65         x = 0;
66         y++;
67     }
68 }

```

之后在 `init_device()` (在 `nanos-lite/src/device.c` 中定义) 中将 `/proc/dispinfo` 的内容提前写入到字符串 `dispinfo` 中：

```

70 void init_device() {
71     _ioe_init();
72
73     // TODO: print the string to array `dispinfo` with the format
74     // described in the Navy-apps convention
75     sprintf(dispinfo, "WIDTH : %d\nHEIGHT: %d\n", _screen.width, _screen.height);
76     dispinfo_size = strlen(dispinfo);
77 }

```

实现 `dispinfo_read()` (在 `nanos-lite/src/device.c` 中定义), 把字符串 `dispinfo` 中 `offset` 开始的 `len` 字节写到 `buf` 中:

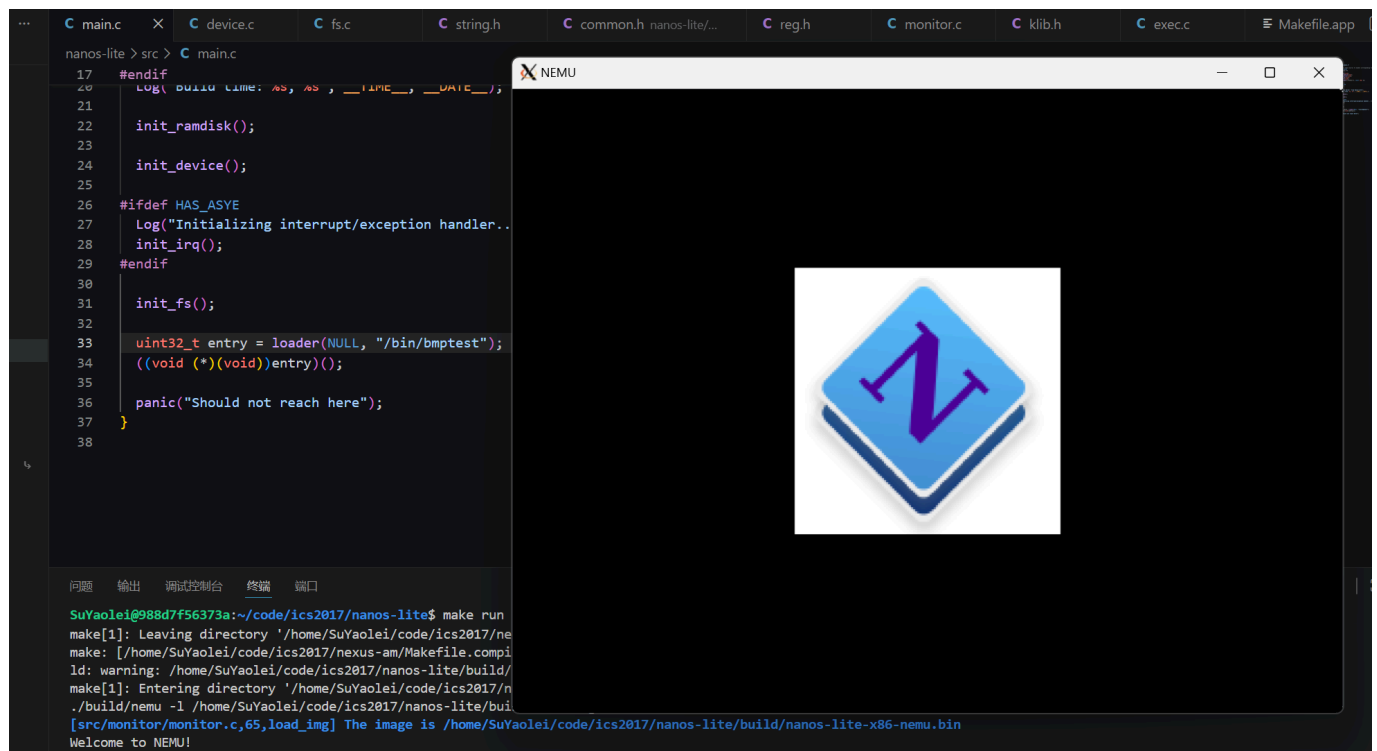
```

37 void dispinfo_read(void *buf, off_t offset, size_t len) {
38     if (offset >= (off_t)dispinfo_size) {
39         return;
40     }
41
42     if (offset + len > dispinfo_size) {
43         len = dispinfo_size - offset;
44     }
45
46     memcpy(buf, dispinfo + offset, len);
47
48 }

```

这里需要在之前实现的 `fs_read()` 中补充对 `/proc/dispinfo` 的支持, 前面已经一并说明, 这里不再赘述。

然后让 Nanos-lite 加载 `/bin/bmptest`, 运行结果如下:



能看到屏幕上显示 ProjectN 的 Logo, 实现没有问题。

10.把设备输入抽象成文件

首先, 要实现 `events_read()` (在 `nanos-lite/src/device.c` 中定义):

```

13 size_t events_read(void *buf, size_t len) {
14     char event[32];
15     int key = _read_key();
16
17     if (key != _KEY_NONE) {
18         const char *type = (key & KEYDOWN_MASK) ? "kd" : "ku";
19         key &= ~KEYDOWN_MASK;
20         sprintf(event, "%s %s\n", type, keyname[key]);
21     }
22     else {
23         sprintf(event, "t %u\n", (unsigned)_uptime());
24     }
25
26     size_t event_len = strlen(event);
27     if (len > event_len) {
28         len = event_len;
29     }
30     memcpy(buf, event, len);
31     return len;
32 }

```

events_read 函数用于读取输入事件或系统时间信息，并将结果写入缓冲区 buf。

调用 _read_key() 获取键盘输入，如果读取到按键事件，则根据 KEYDOWN_MASK 判断是按下事件还是松开事件，然后结合按键名称 keyname[key] 使用 sprintf 生成事件字符串。

如果当前没有键盘事件，则调用 _uptime() 获取系统运行时间，并同样生成时间事件字符串。随后函数计算事件字符串长度 event_len，并取 len 与 event_len 中较小值作为实际拷贝长度，最后通过 memcpy 将事件内容复制到用户缓冲区 buf 中，并返回实际读取的字节数。

之后在 fs_read() 里补充对 /dev/events 的支持，前文已经说过，这里不再赘述。

之后让 Nanos-lite 加载 /bin/events，运行结果如下：

```

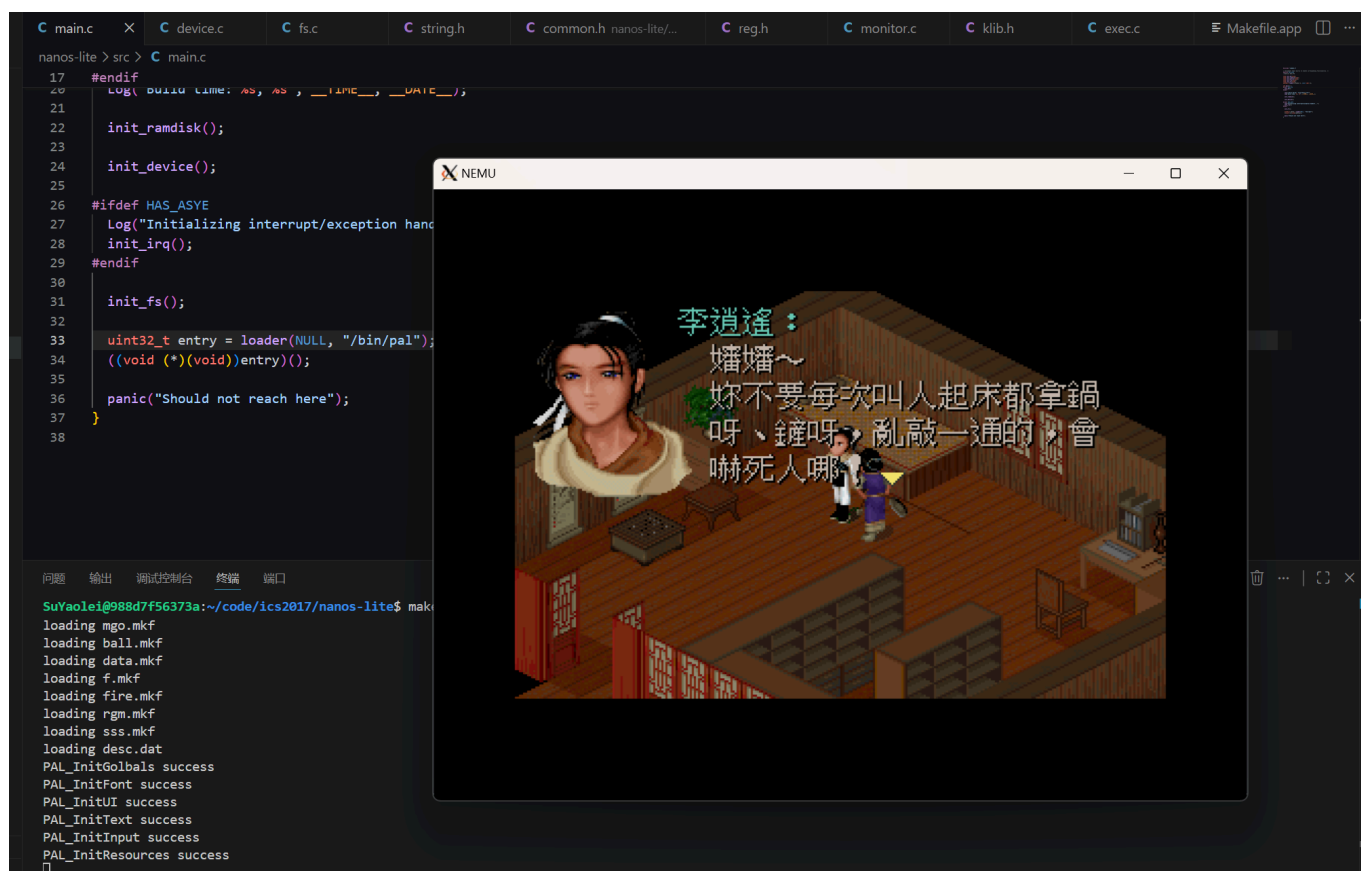
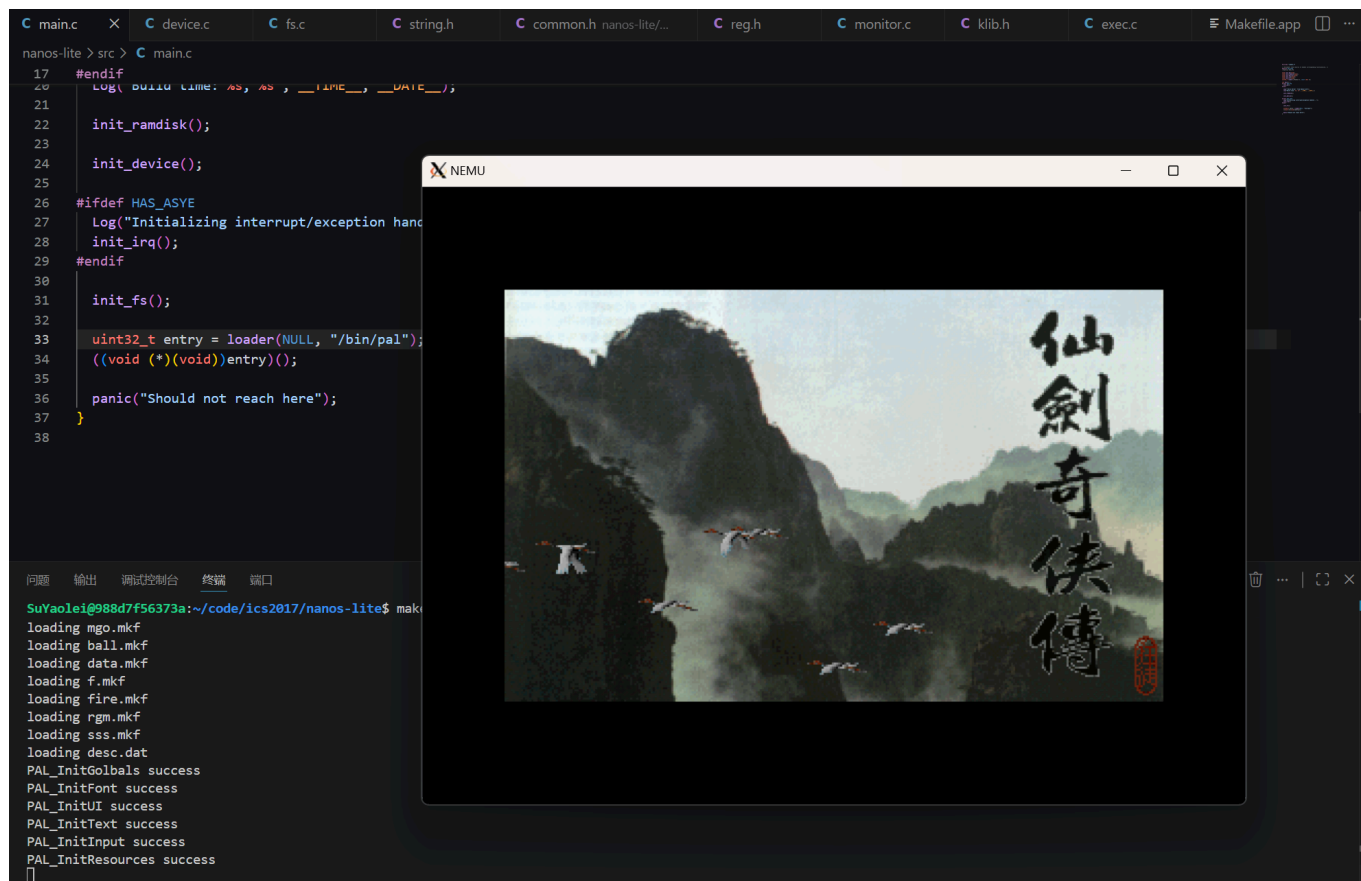
SuYaoLei@988d7f56373a:~/code/ics2017/nanos-lite$ make run
receive event: kd W
receive event: ku W
receive event: t 7399
receive event: kd J
receive event: ku J
receive event: t 7790
receive event: t 7907
receive event: kd S
receive event: ku S
receive event: t 8271
receive event: kd N
receive event: ku N
receive event: t 8634
receive event: t 8776
receive event: t 8917

```

能看到程序输出时间事件的信息，敲击按键时会输出按键事件的信息，实现是正确的。

11.在NEMU中运行仙剑奇侠传

将仙剑奇侠传的数据文件解压并放到 navy-apps/fsimg/share/games/pal/ 目录下，更新 ramdisk 之后，在 Nanos-lite 中加载并运行 /bin/pal：



还是可以把玩一下的。

问题：不再神秘的秘技

网上流传着一些关于仙剑奇侠传的秘技，其中的若干条秘技如下：

1. 很多人到了云姨那里都会去拿三次钱，其实拿一次就会让钱箱爆满！你拿了一次钱就去买剑把钱用到只剩一千多，然后去道士那里，先不要上楼，去掌柜那里买酒，多买几次你就会发现钱用不完了。
 2. 不断使用乾坤一掷(钱必须多于五千文)用到财产低于五千文，钱会暴增到上限，如此一来就有用不完的钱了
 3. 当李逍遥等级到达99级时，用5~10只金蚕王，经验点又跑出来了，而且升级所需经验会变回初期5~10级内的经验值，然后去打敌人或用金蚕王升级，可以学到灵儿的法术(从五气朝元开始)；升到199级后再用5~10只金蚕王，经验点再跑出来，所需升级经验也是很低，可以学到月如的法术(从一阳指开始)；到299级后再用10~30只金蚕王，经验点出来后继续升级，可学到阿奴的法术(从万蚁蚀象开始)。
- 假设这些上述这些秘技并非游戏制作人员的本意，请尝试解释这些秘技为什么能生效。

查阅一些资料，能知道在早期 DOS 游戏中，由于资源有限，开发者通常直接使用固定大小的数据类型比如 8 位或 16 位整数来保存金钱、经验值和等级等数据，当这些数值超过表示范围时，就容易出现异常行为。

“钱用不完”可能是因为金钱使用了无符号整数保存，当程序进行减法或判断时，如果金钱减少到某个临界值以下，发生了下溢，数值会从 0 跳转到该数据类型能够表示的最大值，于是玩家的钱就突然暴增。

类似地，“乾坤一掷”可能会在扣钱时没有正确处理“钱不足”的情况，从而导致数值回绕到上限。

而等级超过 99 后重新出现经验值、升级经验异常降低以及学会其他角色法术等问题，则可能与数组越界或等级数据溢出有关。一些老游戏会用数组按等级存储升级所需经验和技能表，如果程序默认最大等级是 99，但没有严格限制继续升级，那么当等级达到更高数值后，数组索引可能访问到本不属于当前角色的数据区域，于是就会错误地读取到其他角色的技能数据，从而“学会灵儿或月如法术”。同时，等级数据如果超出原本设计范围，也可能导致经验需求表重新从低等级区域开始读取，于是“升级经验重新变低”。

四、必答题

文件读写的具体过程 仙剑奇侠传中有以下行为：

在navy-apps/apps/pal/src/global/global.c的PAL_LoadGame()中通过fread()读取游戏存档
在navy-apps/apps/pal/src/hal/hal.c的redraw()中通过NDL_DrawRect()更新屏幕

请结合代码解释仙剑奇侠传，库函数，libos，Nanos-lite，AM，NEMU是如何相互协助，来分别完成游戏存档的读取和屏幕的更新。

游戏存档读取过程

游戏首先在 PAL_LoadGame() 中调用 fread() 读取存档文件。

```
371 //
372 // Read all data from the file and close.
373 //
374 fread(&s, sizeof(SAVEDGAME), 1, fp);
375 fclose(fp);
```

fread() 属于 C 标准库函数，它负责从 FILE 结构维护的文件缓冲区中读取数据。当缓冲区中的数据不足时，fread() 会调用 __srefill() 重新填充缓冲区，而 __srefill() 底层最终会调用 read() 系统调用。随后，libos 会通过系统调用陷入内核，触发 SYS_read。

Nanos-lite 接收到系统调用后，调用 fs_read() 在文件系统中读取对应文件的数据。ramdisk_read() 又会通过 AM 提供的接口访问物理内存，而这些内存最终由 NEMU 模拟，最终成功将游戏存档数据读入用户缓冲区。

屏幕更新过程

首先游戏会在 redraw() 中先根据调色板将游戏虚拟显存 vmem 转换为真实颜色值，并写入帧缓冲数组 fb 中。

```
96 static void redraw() {
97     for (int i = 0; i < W; i++)
98         for (int j = 0; j < H; j++)
99             fb[i + j * W] = palette[vmem[i + j * W]];
100
101     NDL_DrawRect(fb, 0, 0, W, H);
102     NDL_Render();
103 }
```

随后调用 NDL_DrawRect() 将像素数据写入 NDL 的画布 canvas。接着 NDL_Render() 会逐行调用库函数 fseek() 和 fwrite() 向帧缓冲设备文件 fbdev 写入像素数据。

```
51 int NDL_DrawRect(uint32_t *pixels, int x, int y, int w, int h) {
52     if (has_nwm) {
53         for (int i = 0; i < h; i++) {
54             printf("\033[X%d;Y%d", x, y + i);
55             for (int j = 0; j < w; j++) {
56                 putchar(' ');
57                 fwrite(&pixels[i * w + j], 1, 4, stdout);
58             }
59             printf("d\n");
60         }
61     } else {
62         for (int i = 0; i < h; i++) {
63             for (int j = 0; j < w; j++) {
64                 canvas[(i + y) * canvas_w + (j + x)] = pixels[i * w + j];
65             }
66         }
67     }
68 }
69
70 int NDL_Render() {
71     if (has_nwm) {
72         fflush(stdout);
73     } else {
74         for (int i = 0; i < canvas_h; i++) {
75             fseek(fbdev, ((i + pad_y) * screen_w + pad_x) * sizeof(uint32_t), SEEK_SET);
76             fwrite(&canvas[i * canvas_w], sizeof(uint32_t), canvas_w, fbdev);
77         }
78         fflush(fbdev);
79     }
80 }
```

这里的 fwrite() 最终会触发 write 系统调用，进入 libos，再进入 Nanos-lite 的 SYS_write 处理逻辑。Nanos-lite 发现目标文件是 FD_FB 后，会调用 fb_write() 将数据写入显存设备。fb_write() 底层通过 AM 的接口将像素数据写入显存，而 AM 最终会调用 NEMU 模拟的设备模型更新窗口显示内容。

五、实验过程中的问题与解决方法

在最开始编译 dummy 程序的时候我执行 make，会报错有一个函数未定义，我本以为是上次实验有什么地方没有实现，但是询问助教老师后得到回复是 gcc 的版本太高了，我的 gcc 版本是 14，建议我将版本降低到 5.4，于是经过一段时间的摸索（我是 docker 的 debian 环境），我终于成功把版本降下来了，下面是一篇我个人书写的经验帖：

[Docker Debian 环境 GCC 14.0.2 回退到 GCC 5.4 完整教程](#)

除此之外，对于 /dev/events 的时间事件，在 events_read() 里我最开始使用长整型格式：

```
23      sprintf(event, "t %lu\n", _uptime());
```

运行 /bin/events 输出错误的内容，如下：

```
SuYaoLei@988d7f56373a:~/code/ics2017/nanos-lite$ make run
ld: warning: /home/SuYaoLei/code/ics2017/nanos-lite/build/nanos-lite-x86-nemu has a LOAD segment with RWX permissions
make[1]: Entering directory '/home/SuYaoLei/code/ics2017/nemu'
./build/nemu -l /home/SuYaoLei/code/ics2017/nanos-lite/build/nemu-log.txt /home/SuYaoLei/code/ics2017/nanos-lite-x86-nemu.bin
[src/monitor/monitor.c,65,load_img] The image is /home/SuYaoLei/code/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 17:06:44, May 18 2026
For help, type "help"
(nemu) c
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 19:47:36, May 18 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x102480, end = 0x1d510c5, size = 29682757 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
receive event: t u
receive event: t u
receive event: t u
receive event: t u
receive event: t u
receive event: t u
```

最终改成整型格式即可修正时间事件的输出：

```
23      sprintf(event, "t %u\n", (unsigned)_uptime());
```

当然，在实现一个函数或者指令之后，比如 fs_read() 和 int 等等，别忘了在相应位置补上对应函数的声明...

总之，本次实验最大的问题还是最开始的要回退 gcc 版本，整个 PA3 做下来还是让我收获颇丰。